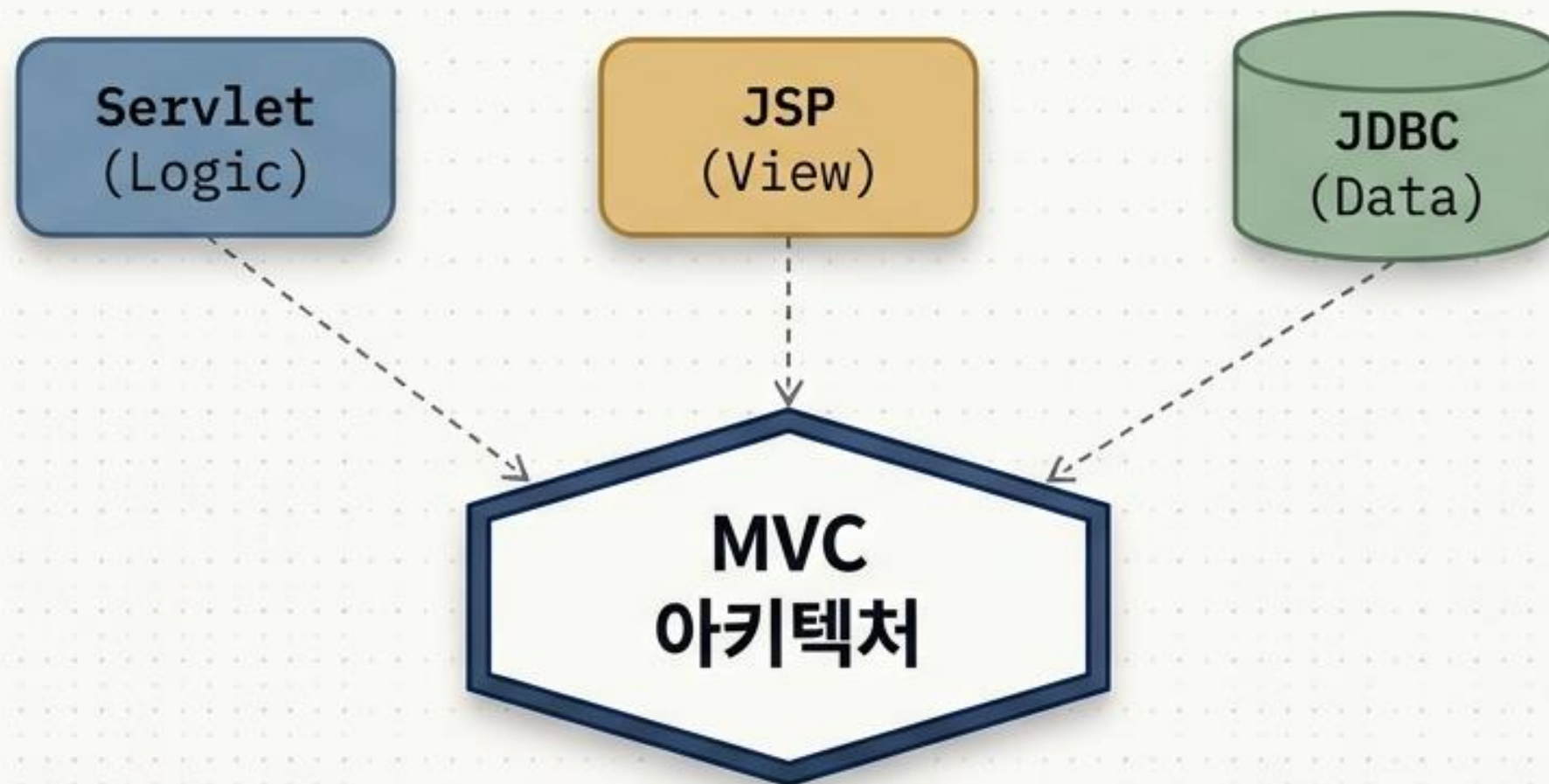


Java 웹 서버사이드 아키텍처

JSP, Servlet, JDBC부터 MVC 패턴까지: 시스템 청사진



JSP & Servlet: 자바 웹 개발의 핵심 아키텍처와 MVC 패턴 이해하기

JSP vs Servlet: 역할과 실행 원리



Servlet은 '로직' 중심
Servlet은 비즈니스 로직을 처리하는 Controller 역할 담당



JSP는 '화면' 중심
JSP는 결과를 출력하는 View 역할 담당

JSP는 결국 Servlet으로 변환되어 실행



JSP 파일은 실행 시 웹 컨테이너에 의해 .java 소스와 .class 파일로 컴파일되어 동작

속도 최적화를 위한 3단계 과정



과정 거치고, 이후에는 메모리 내 객체 재사용

MVC 패턴의 데이터 흐름(DTO, DAO, JDBC)



사용자 요청



Controller (Servlet)



DTO는 '데이터 바구니' DAO는 'DB 전문가'
DTO는 계층 간 데이터 전달을 위한 객체
DAO는 DB 접근 및 CRUD 로직 전담



페이지 이동 방식: Forward vs Redirect 차이

Forward (포워드)	Redirect (리다이렉트)
URL 변화 변하지 않음 (서버 내부 이동)	새로운 주소로 변경됨
데이터 유지 request 객체와 데이터 유지됨	데이터 유지 새로운 요청으로 데이터 초기화됨
주요 용도 결과 화면 출력 (조회 등)	주요 용도 상태 변경 후 페이지 전환 (로그인 등)

JDBC 데이터 처리 6단계 표준



SQL Injection 방지를 위한 PreparedStatement



쿼리 구조 & 데이터 분리
보안 강화, 실행 성능 높이는 실무 필수 기술



최종 응답 화면

JSP & Servlet: 현대 웹 개발의 핵심 아키텍처 이해하기

JSP/Servlet의 동작 원리와 차이점



JSP는 실행 시 자동으로 Servlet으로 변환됩니다.



JSP 요청 시 'JSP → Java(.java) → Class(.class)' 과정을 거쳐 서블릿으로 실행됩니다.

Servlet 생명주기: 생성부터 소멸까지

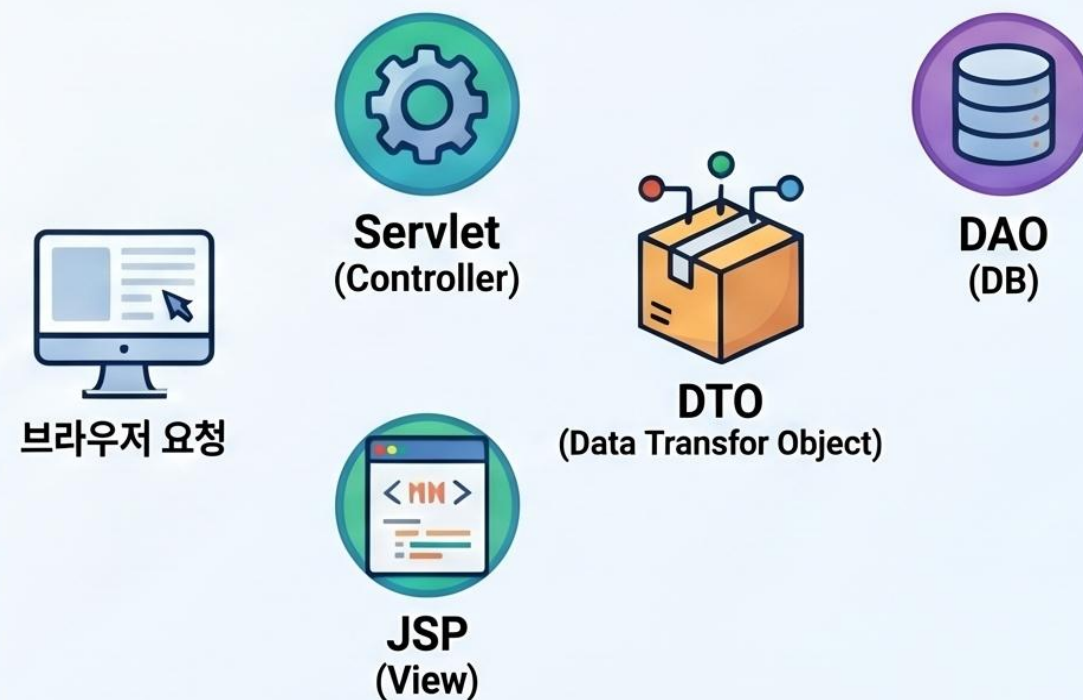


Servlet은 로직 중심, JSP는 화면 중심



서블릿은 Controller 역할을, JSP는 View 역할을 수행하여 코드의 혼잡을 방지합니다.

실무 표준 아키텍처 (MVC 패턴과 DTO)



DTO(Data Transfer Object)의 필요성
흩어진 데이터를 하나의 객체로 묶어 계층 간에 안전하고 편리하게 전달합니다.



JSP 내 로직 처리(Scriptlet) 지양

JSP 안에서러 Java 코드 사용을 줄이고 EL(S())과 JSTL 태그 사용을 권장합니다.

아키텍처의 양대 기둥: Servlet vs JSP

Servlet (Controller)

- Java 중심의 로직 처리
- 확장자: .java

```
out.println("<h1>Hello</h1>");
```

UI 작성이 매우 불편함

JSP (View)

- HTML 중심의 화면 출력
- 확장자: .jsp

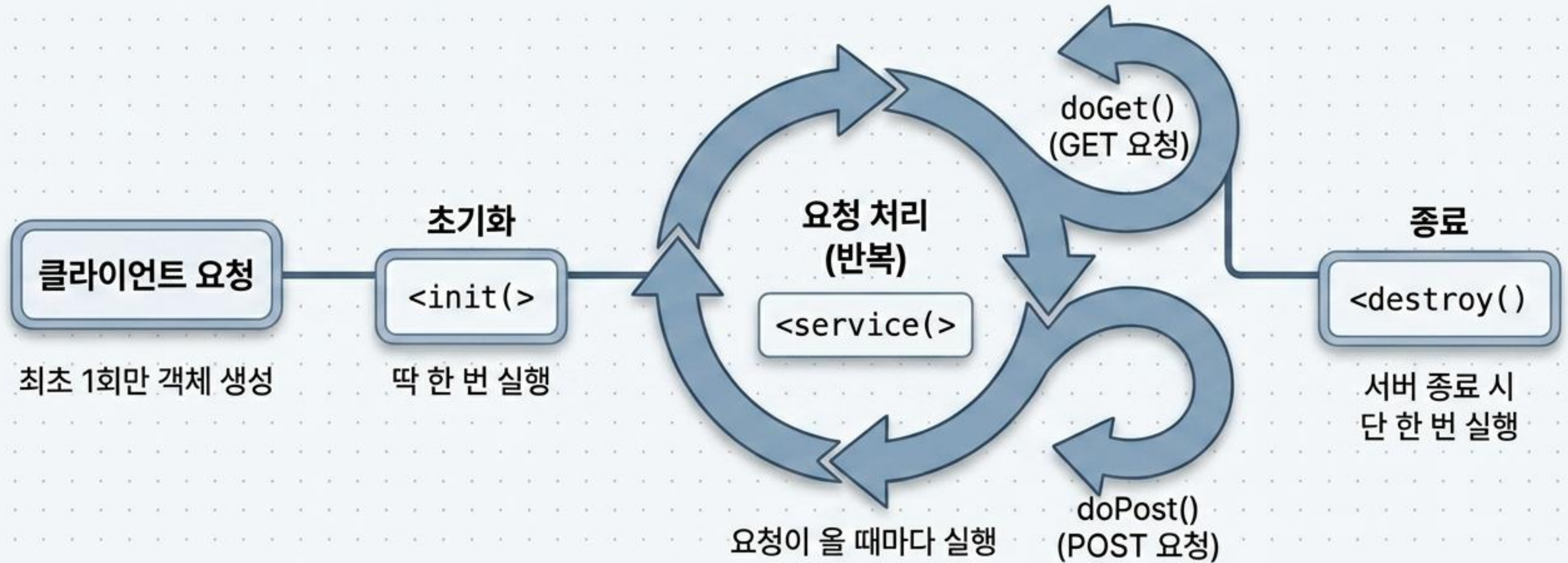
```
<h1>Hello <%= name %></h1>
```

직관적인 UI 작성 가능

핵심 진실: JSP는 결국 Servlet이다



Servlet 생명주기 (Lifecycle)



JSP 문법의 해부학: 자동 변환의 원리

page (페이지 속성)

include (정적 포함)

taglib (JSTL 선언)

JSP 스크립트 문법

변환된 Servlet Java 코드

Scriptlet <% %>

<% int num = 10; %>

service() 내부에 일반 코드로 삽입

int num = 10;

Expression <%= %>

<%= new Date() %>

세미콜론 금지

자동으로 출력 코드로 변환

out.print(new Date());

Declaration <%! %>

<%! String name; %>

Servlet 클래스의 멤버 변수로 선언

String name;

JSP 내장 객체 대시보드

JSP가 Servlet으로 변환될 때 톰캣이 자동으로 생성해주는 핵심 객체들

request

클라이언트 요청 정보

Type: `HttpServletRequest`

Methods: `getParameter()`,
`setAttribute()`

response

서버의 응답 처리

Type: `HttpServletResponse`

Methods: `sendRedirect()`,
`addCookie()`

session

사용자별 상태 유지 (로그인)

Type: `HttpSession`

Methods: `setAttribute()`,
`invalidate()`

application

웹 애플리케이션 전체 공유 공간

Type: `ServletContext`

out

화면 출력 스트림

Type: `JspWriter`

Methods: `print()`, `flush()`

상태 관리 메커니즘: Session vs Cookie

Session

서버 메모리에 저장

- 강력한 보안성 (클라이언트는 Session ID만 소유)
- 모든 Java 객체 저장 가능
- 용도: 로그인 상태 유지
- 종료:
`session.invalidate()`
호출 시

←-----
서버가 Set-Cookie로 응답
-> 브라우저는 다음 요청 시
자동 전송

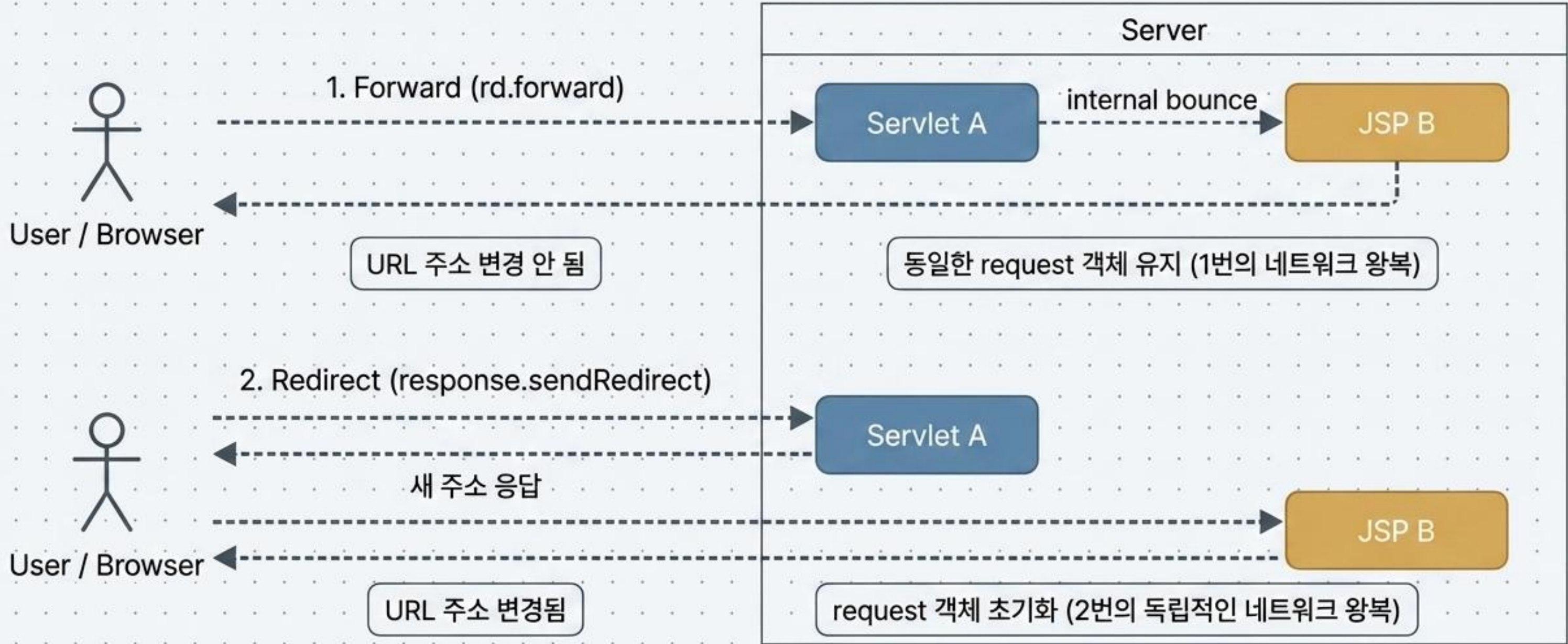
-----→

Cookie

클라이언트 브라우저에 저장

- 낮은 보안성
(사용자에게 노출됨)
- 문자열 데이터만 저장 가능
- 용도: 장바구니, 팝업
'오늘 하루 보지 않기'
- 수명: `setMaxAge()`로
유효시간 설정

페이지 이동 논리: Forward vs Redirect



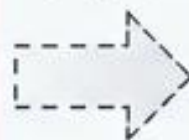
URL 유지 여부	Forward	Redirect
Request 객체 유지 여부	(O, O)	(X, X)

JSP의 현대화: Java 코드 분리 (EL & JSTL)

Code Refactoring

❌ 과거: 디자인과 로직의 혼재 (유지보수 지옥)

```
<% for(int i=0; i<list.size(); i++) { %>
    <%= list.get(i) %>
<% } %>
```



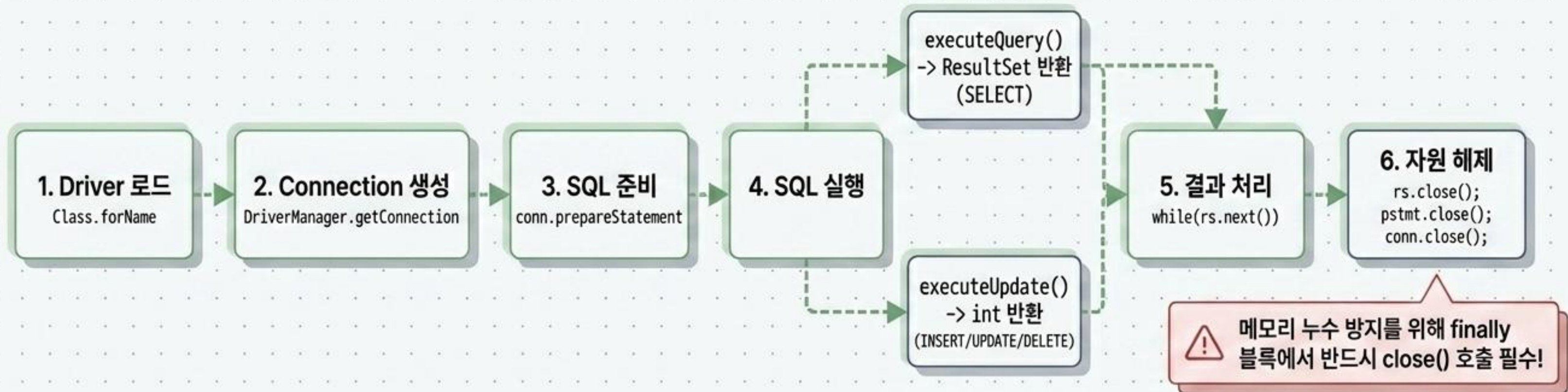
○ 현재: 값 출력은 EL로, 제어문은 JSTL로 분리

```
<c:forEach var="item" items="${list}">
    ${item}
</c:forEach>
```



실무 핵심 규칙: JSP 파일 내부에서는 Scriptlet(<% %>)을 거의 사용하지 않으며, 오직 View(화면 렌더링)의 역할만 담당해야 한다.

데이터 파이프라인: JDBC 실행 흐름



JDBC의 엔터프라이즈 최적화: 보안과 성능

PreparedStatement (보안 및 SQL Injection 차단)

과거: 보안 취약 (SQL Injection)

 "select * from user where id='"+id+"'" data-bbox="159 313 462 343"/>

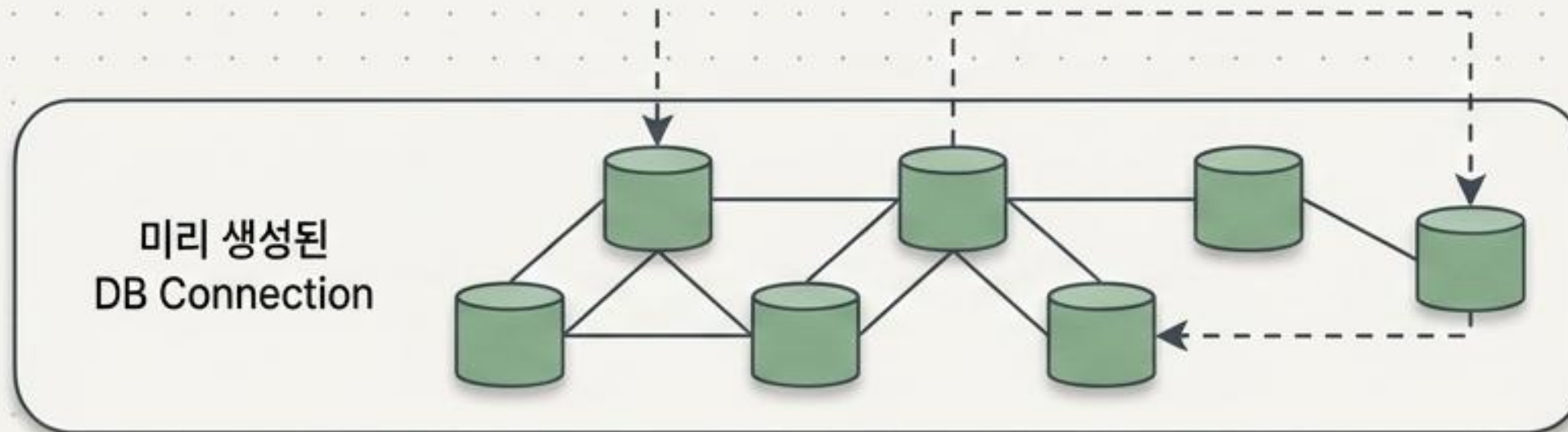
현재: 보안 강화 (PreparedStatement)

select * from user where id=?
pstmt.setString(1, "admin")



쿼리를 미리 컴파일하고 '?' 바인딩을 통해 악의적인 SQL Injection 공격을 원천 차단하며 쿼리 재사용성을 높임.

DB Connection Pool / DBCP (성능 최적화)



Connection 객체 생성 비용은 매우 높음. 미리 여러 개의 Connection을 생성해 Pool에 담아 두고 빌려주고 반납받는 구조로 서버 부하를 방지. (대표 기술: HikariCP, Tomcat DBCP)

데이터 컨테이너 패턴: DTO vs VO

웹 개발에서는 개별 변수의 파편화를 막기 위해 데이터를 객체(JavaBean) 형태로 포장하여 계층 간에 전달합니다.

DTO (Data Transfer Object)



- 목적: 계층 간(Controller ↔ Service ↔ DAO) 데이터 이동
- 특징: 로직이 없고 getter/setter로 구성
- 속성: 데이터 변경 가능 (Mutable)
- 비유: 데이터를 나르는 택배 상자

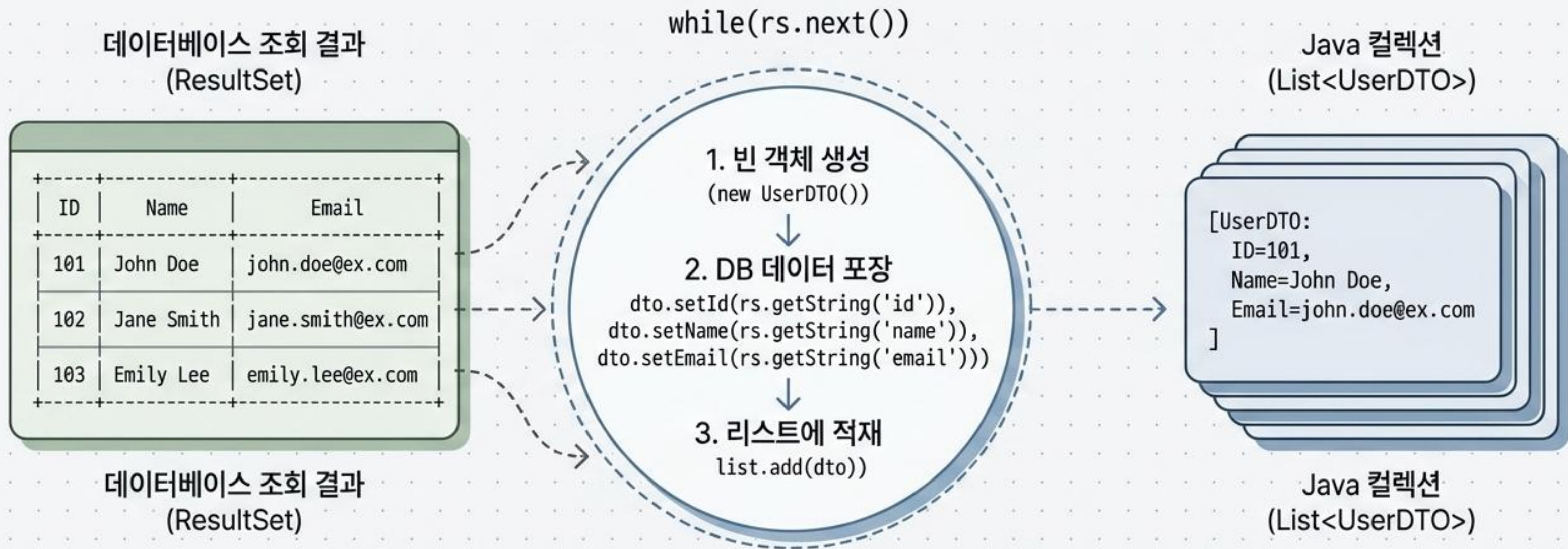
VO (Value Object)



- 목적: 값 자체를 표현
- 특징: 값의 동일성 (equals/hashCode) 비교가 핵심
- 속성: 불변성 지향 (Immutable, 보통 setter 없음)
- 비유: 고유한 가치를 지닌 지폐

💡 실무 팁: JSP/Servlet 프로젝트에서는 DTO와 VO를 혼용하여 사용하는 경우가 많으며, Lombok(@Getter, @Setter)을 통해 반복적인 코드를 제거합니다.

ResultSet에서 DTO 리스트로의 매핑



➡ Servlet은 이 깔끔하게 포장된 단일 List 객체 하나만 `request.setAttribute()`에 담아 JSP로 전달합니다.

아키텍처의 안티 패턴: 스파게티 코드

JSP 파일

```
<div>  
  <div>  
    <table>  
      <% Connection conn = null; %>  
      <% while(rs.next()) {  
        ...  
      } %>  
      SELECT * FROM Users WHERE id = ...  
    </table>  
  </div>  
</div>
```

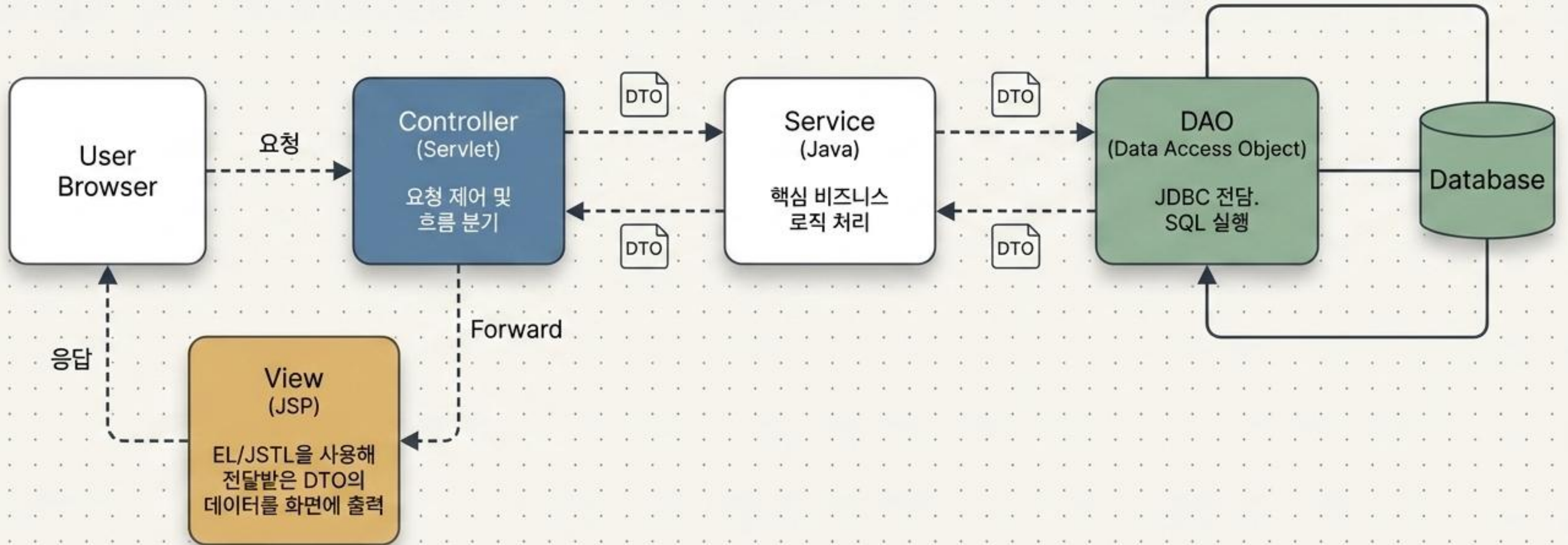
디자인(HTML)과 비즈니스
로직(DB 접속)의 혼재

기능 단위의
재사용 불가능

에러 추적 및
유지보수 불가능

**실무 절대 규칙: JSP 파일 내부에서 DB에 직접 접속하거나 비즈니스 로직을 처리해서는 안 된다.
JSP는 오직 화면(View)만을 렌더링해야 한다.**

완벽한 분업화: 실무 핵심 MVC 아키텍처

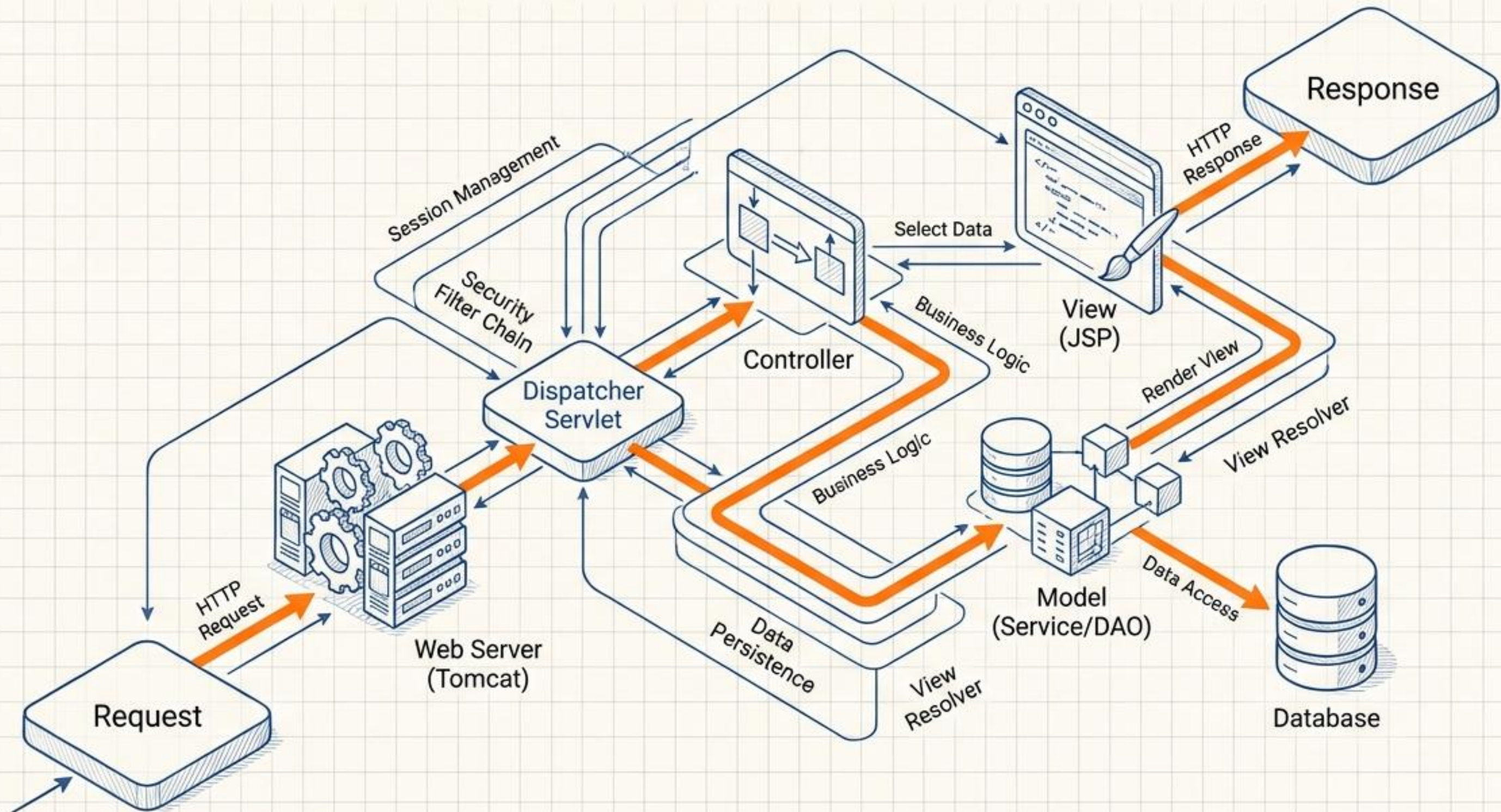


현대적 맥락과 핵심 요약 (Cheat Sheet)



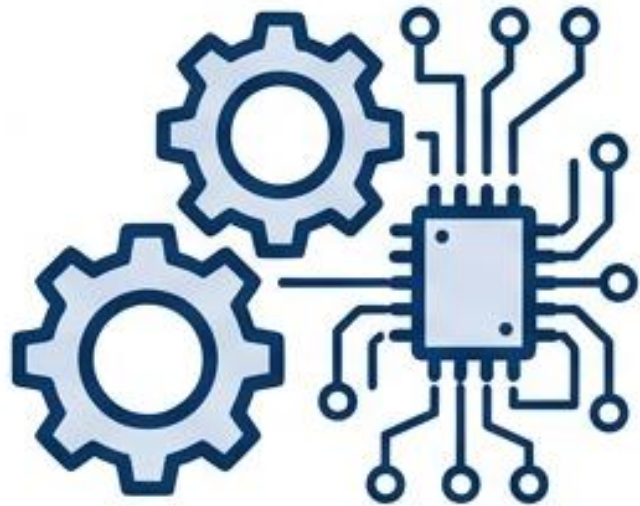
자바 웹 아키텍처 청사진: 파이프라인의 해부학

JSP, Servlet, 그리고 실무 MVC 패턴의 완성 (From Request to Response)



코어 엔진의 이중성: 로직의 제어탑 vs 화면의 쇼룸

Servlet (제어탑)



중심 요소	Java 코드 중심
주요 목적	로직 처리 (Controller 역할)
작성 방식	직접 Java 파일을 작성하고 컴파일
특징	복잡한 로직에 강하나 UI 작성이 매우 불편함

JSP (쇼룸)



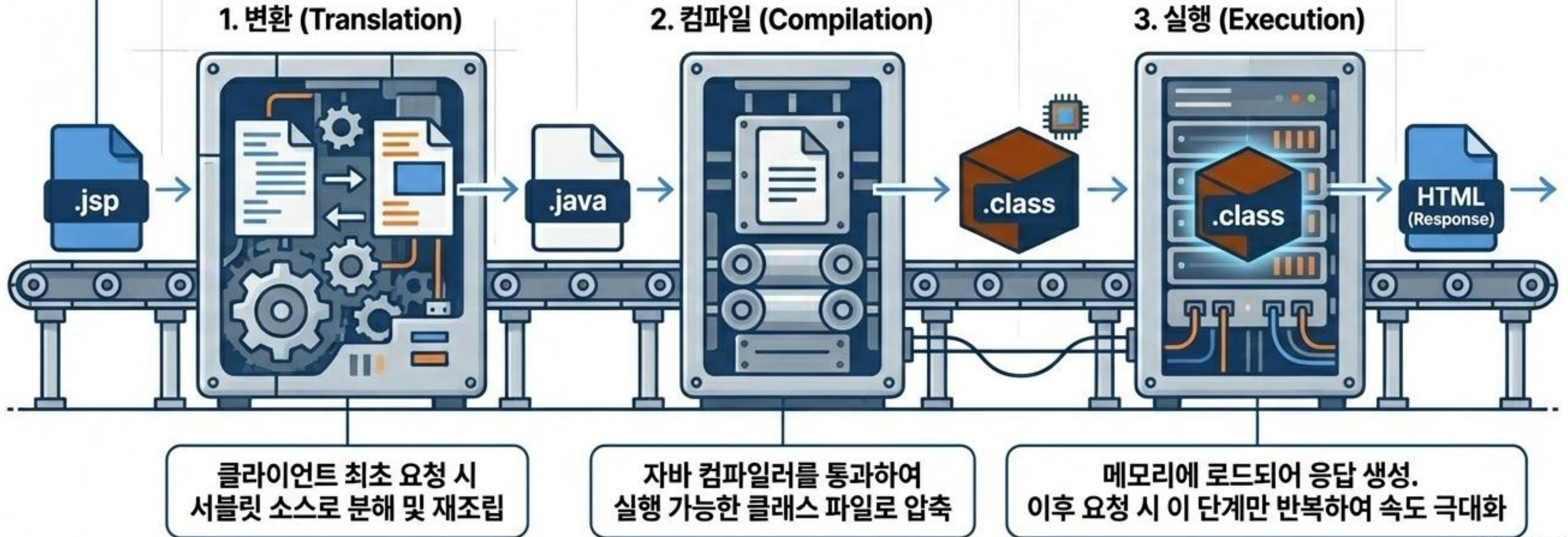
중심 요소	HTML 코드 중심
주요 목적	화면 출력 (View 역할)
작성 방식	HTML 내에 Java 코드를 삽입
특징	UI 작성이 쉽지만 복잡한 로직 관리가 어려움

숨겨진 진실: JSP는 실행 시 웹 컨테이너에 의해 결국 Servlet으로 변환되어 구동됩니다.

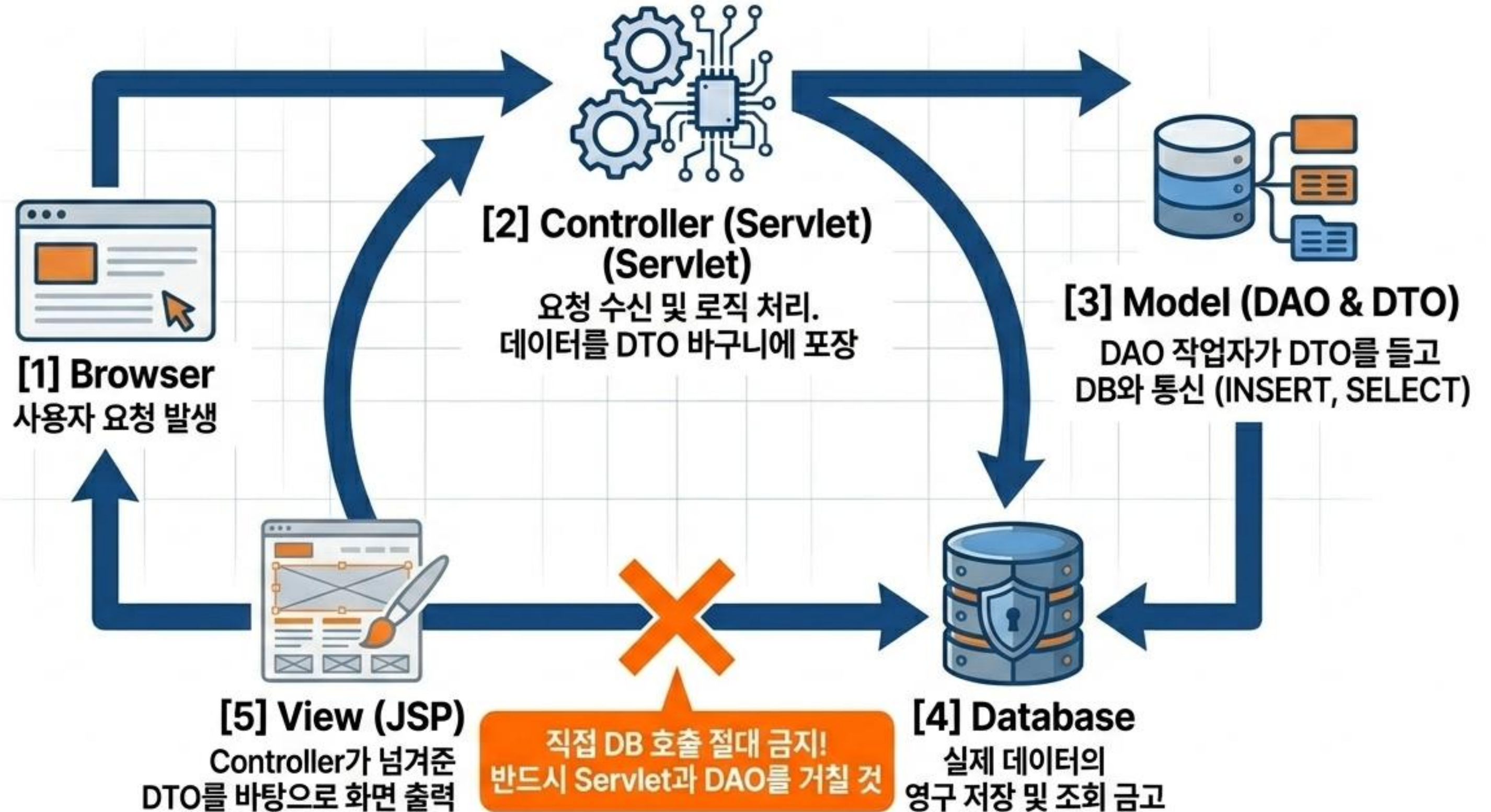
숨겨진 변환 공정: JSP는 어떻게 기계어로 압축되는가

JSP 지시자의 역할 (Directive)

<%@ page %>, <%@ include %>, <%@ taglib %>
변환 공정의 레시피 역할을 하며 전체 페이지 속성을 정의함

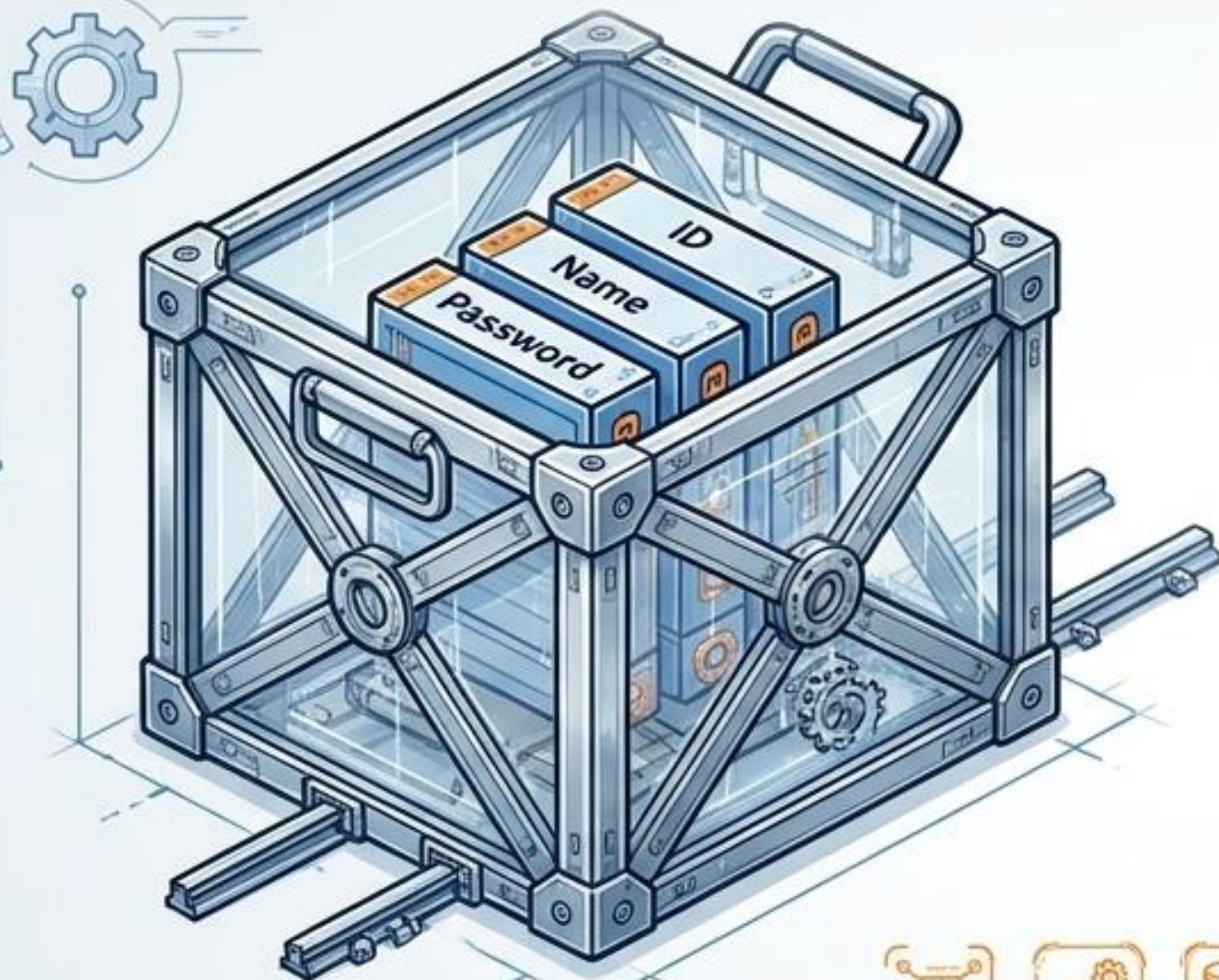


아키텍처의 완성: 실무 MVC 패턴 파이프라인



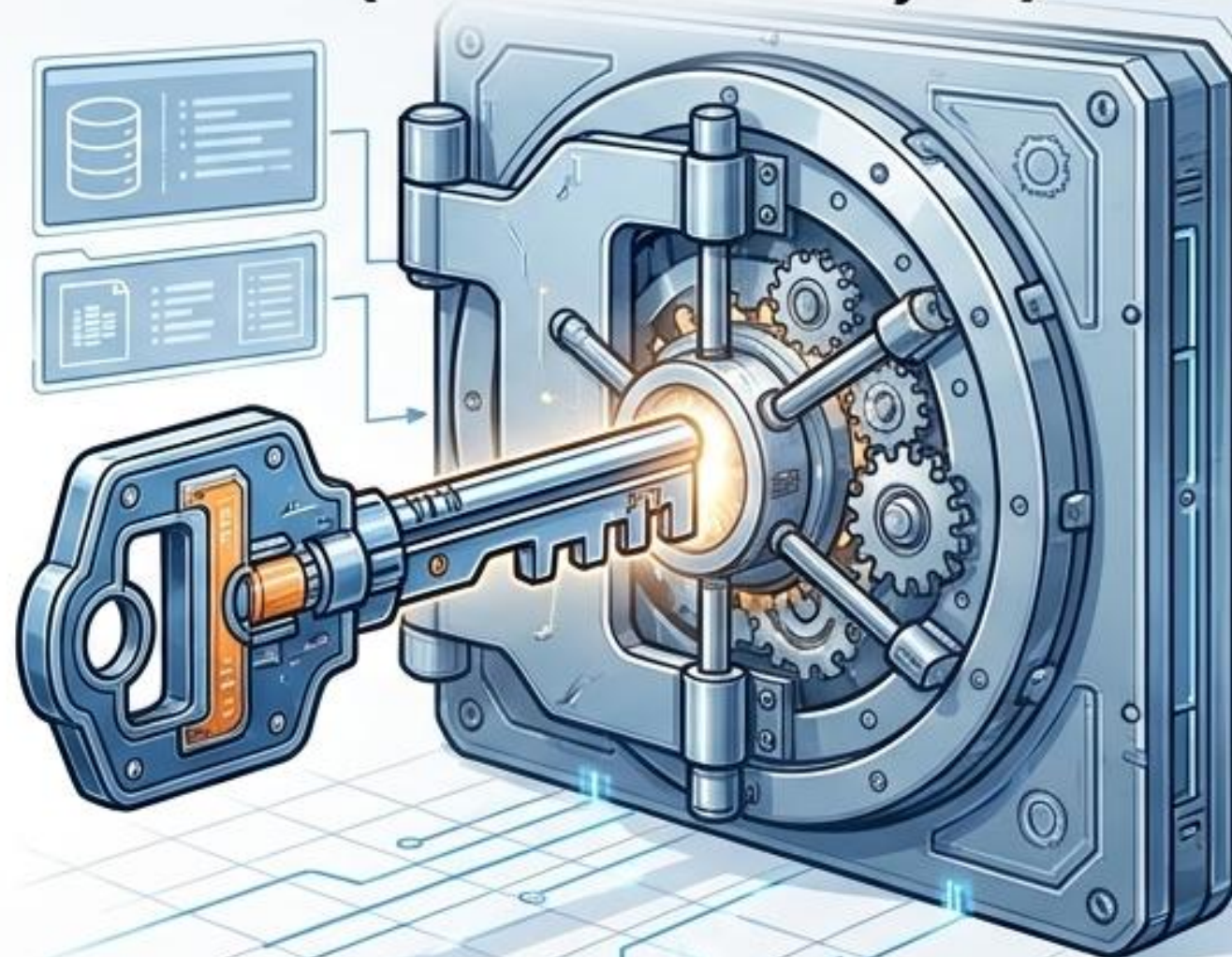
데이터 운반책과 DB 마스터 키: DTO & DAO

DTO (Data Transfer Object)



- 역할** 계층 간 (Controller ↔ DAO ↔ JSP) 데이터를 이동시키는 바구니
- 특징** 로직 없이 private 필드와 getter/setter로만 구성
- 황금 원칙** 비즈니스 로직 포함 절대 금지 (순수 운반 목적)

DAO (Data Access Object)



- 역할** 데이터베이스(DB)와의 직접적인 통신(CRUD)을 전담
- 특징** Connection, PreparedStatement 등을 사용하여 SQL 쿼리 실행
- 황금 원칙** 비즈니스 로직과 SQL을 철저히 분리하여 재사용성 극대화

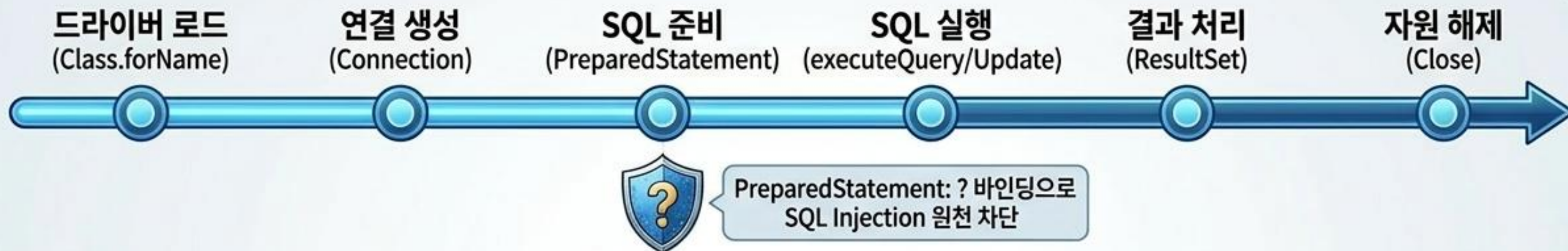
변하지 않는 값의 본질: DTO vs VO

	DTO (Data Transfer Object)	VO (Value Object)
성질		
성질	가변성 (Mutable)	불변성 (Immutable)
설계	setter를 통해 데이터를 채우거나 자유롭게 수정 가능	private final 선언. 생성자로만 값을 설정하며 setter 없음
목적	계층 간 데이터 이동 및 화면 출력	시스템 내에서 고유한 '값' 자체를 표현 (예: 금액, 좌표)

핵심 필수 규칙: VO는 주소가 달라도 값이 같으면 동일 취급해야 하므로, 반드시 equals()와 hashCode()를 오버라이딩해야 합니다.

DB 통신 규약: JDBC 6단계 관문과 Connection Pool

JDBC 6단계 관문 (Stepping Stones)



Connection Pool의 톨게이트 비유

A

✗ 비효율적 방식



매번 다리 건설: 요청마다 Connection을 생성하고 폐기 (자원 낭비, 서버 부하)

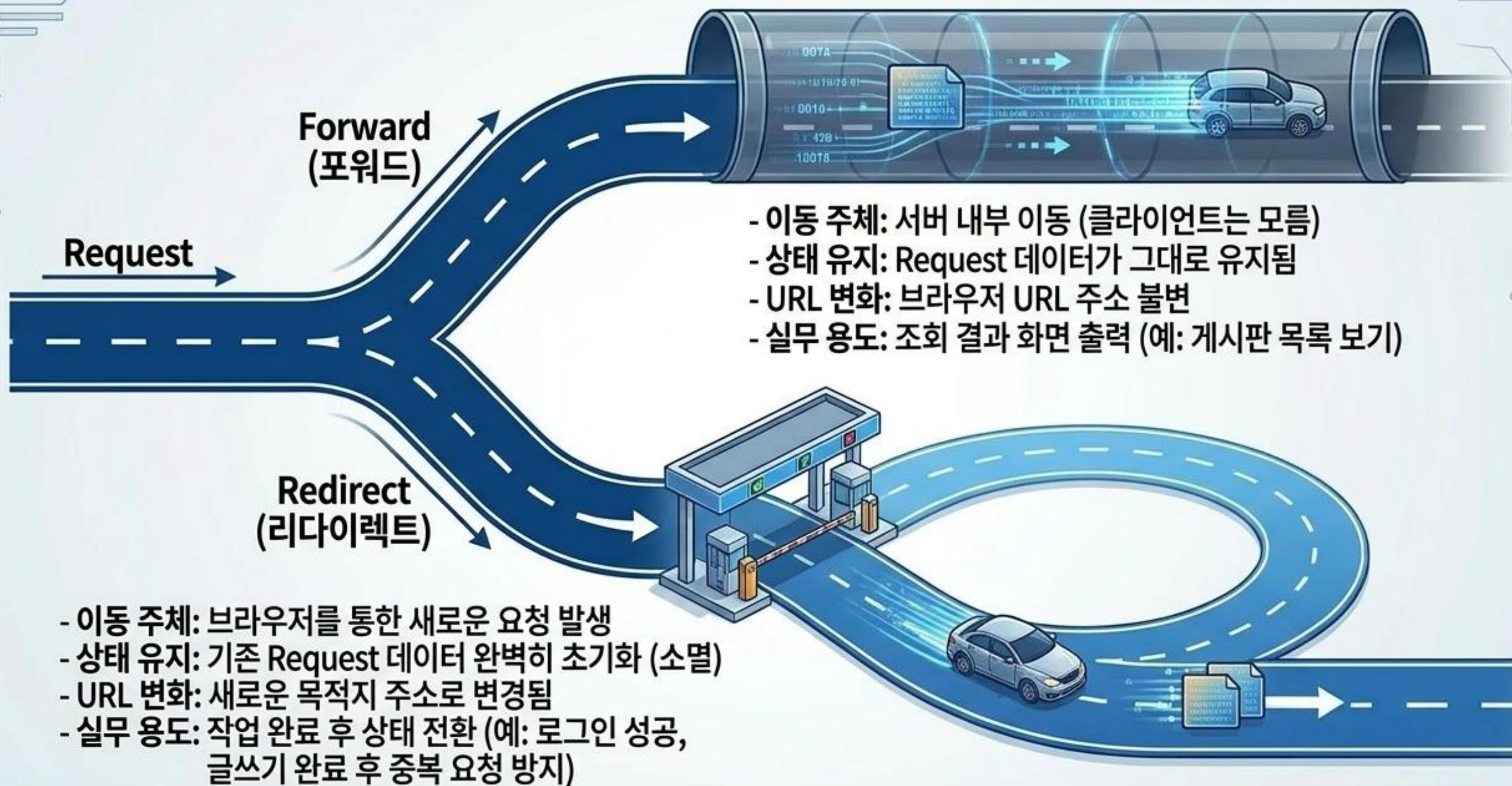
B

✓ DBCP / HikariCP 도입



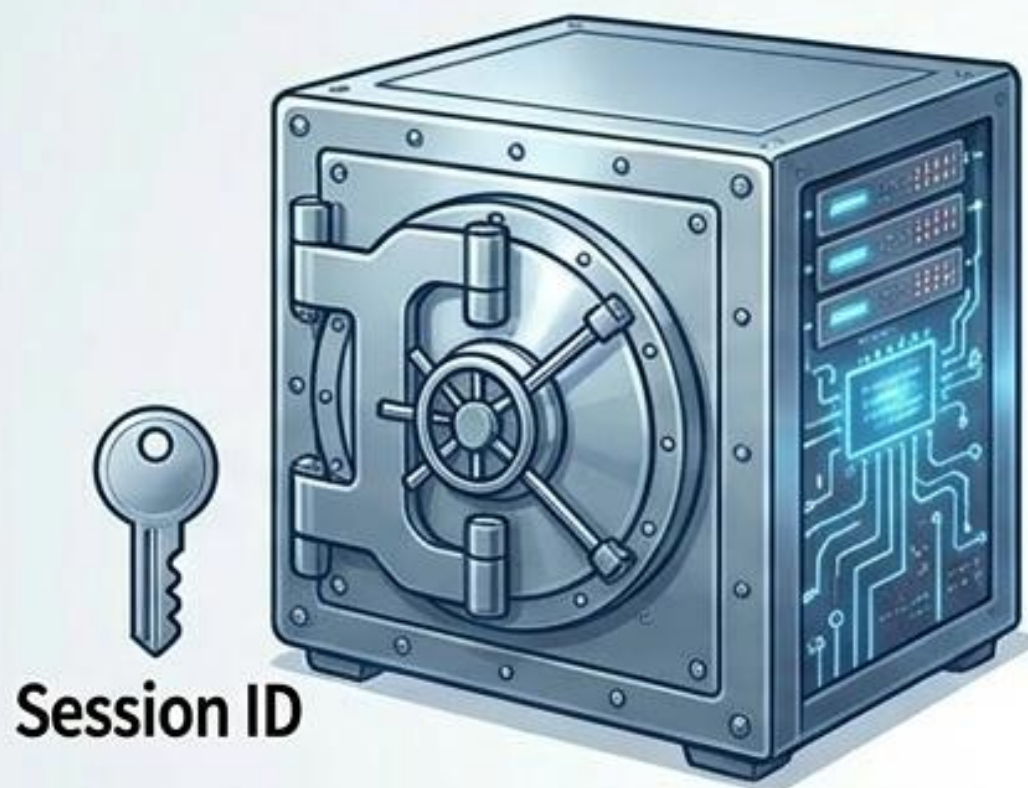
미리 지어진 다리 대여: Pool에 연결을 미리 생성해두고 대여 및 반납 (성능 극대화)

트래픽의 방향타: Forward vs Redirect



사용자를 기억하는 공간: Session vs Cookie

Session (세션)



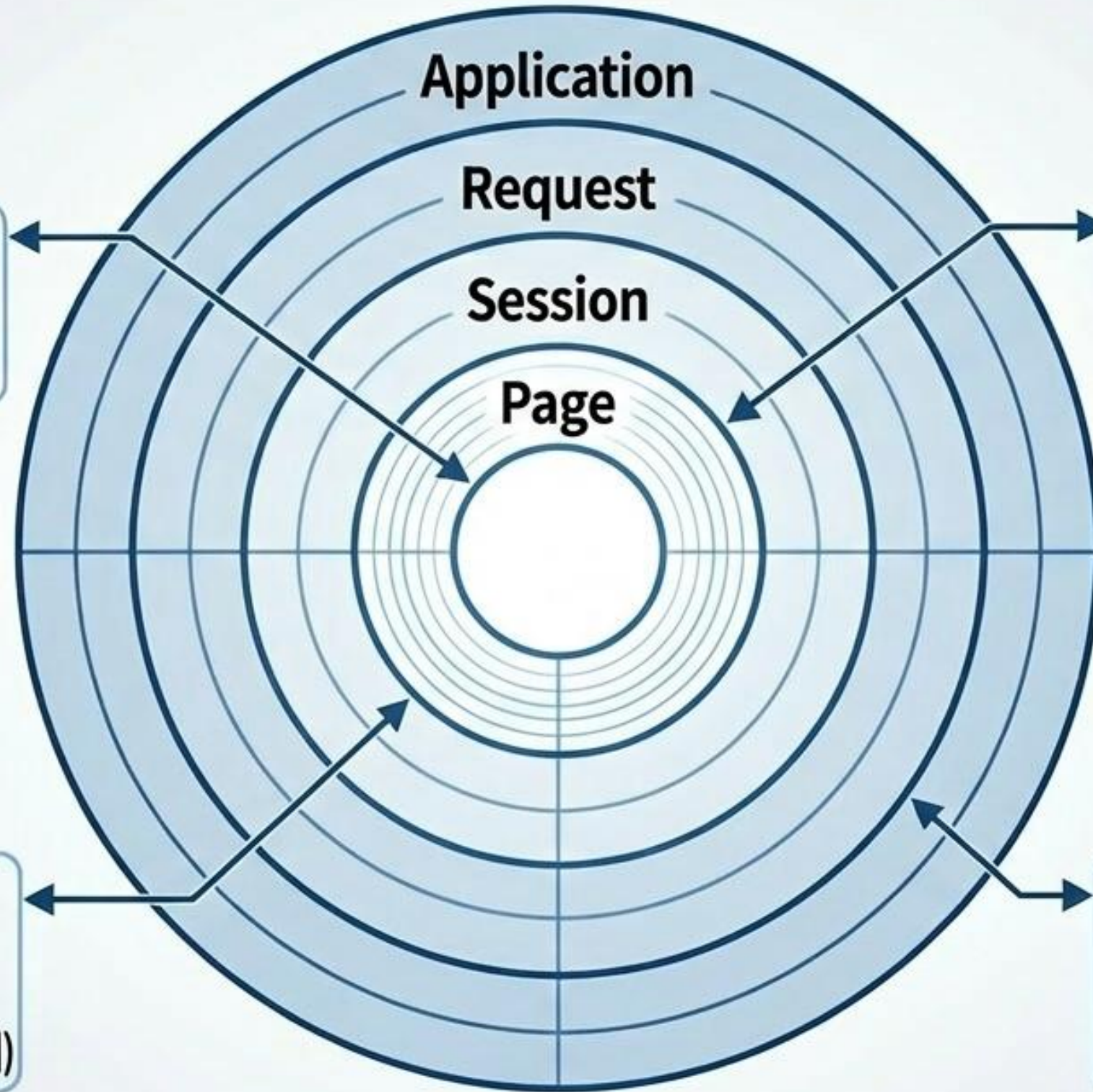
- 저장 위치: 서버 메모리 (안전한 요새)
- 보안 강도: 매우 강함 (데이터가 서버 내부에 격리됨)
- 데이터 형식: 모든 자바 객체(Object, DTO 등) 저장 가능
- 실무 용도: 로그인 상태 유지, 권한 인증 등 필수 보안 기능

Cookie (쿠키)



- 저장 위치: 클라이언트 브라우저 (개인 수첩)
- 보안 강도: 매우 약함 (탈취 및 위변조 위험 노출)
- 데이터 형식: 오직 문자열(String)만 저장 가능
- 실무 용도: 아이디 기억하기, 팝업창 보지 않기 등 단순 편의 기능

데이터의 생명주기: JSP 4대 내장 객체 Scope



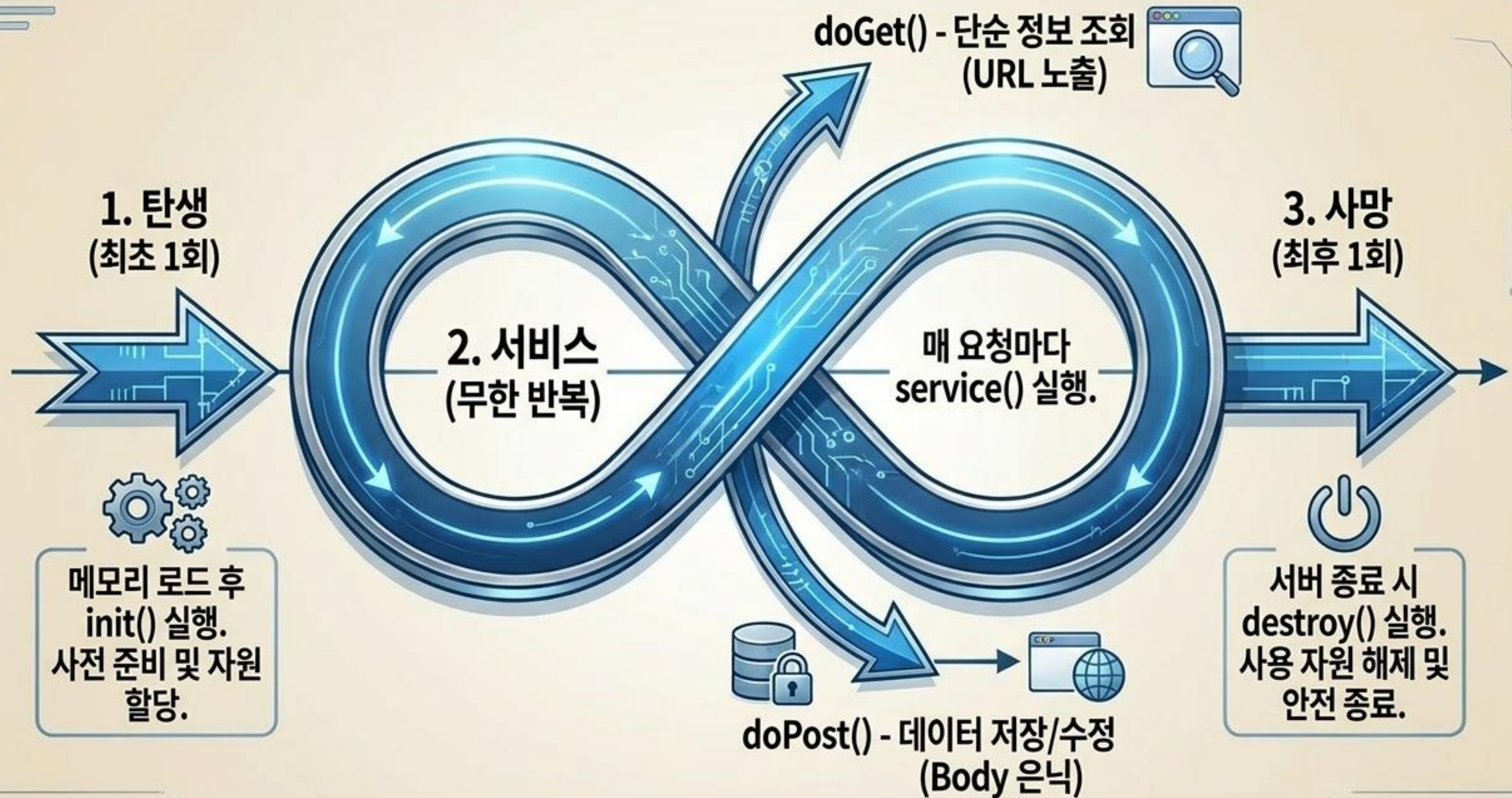

- 객체 : `pageContext`
- 생명주기: 현재 실행 중인 JSP 페이지 내에서만 생존


- 객체 : `request`
- 생명주기: 하나의 요청이 들어와 응답이 나갈 때까지 유지 (Forward 시 공유됨)


- 객체 : `session`
- 생명주기: 사용자 브라우저가 종료되거나 로그아웃 전까지 유지 (개인별 격리)


- 객체 : `application`
생명주기: 서버가 시작되어 종료될 때까지 전체 애플리케이션에서 공용으로 공유

서블릿의 생명주기 (Lifecycle): 단 한 번의 탄생과 무한한 서비스



검문소의 설계: Filter를 통한 공통 로직 제어

필터 3대 임무 (doFilter)

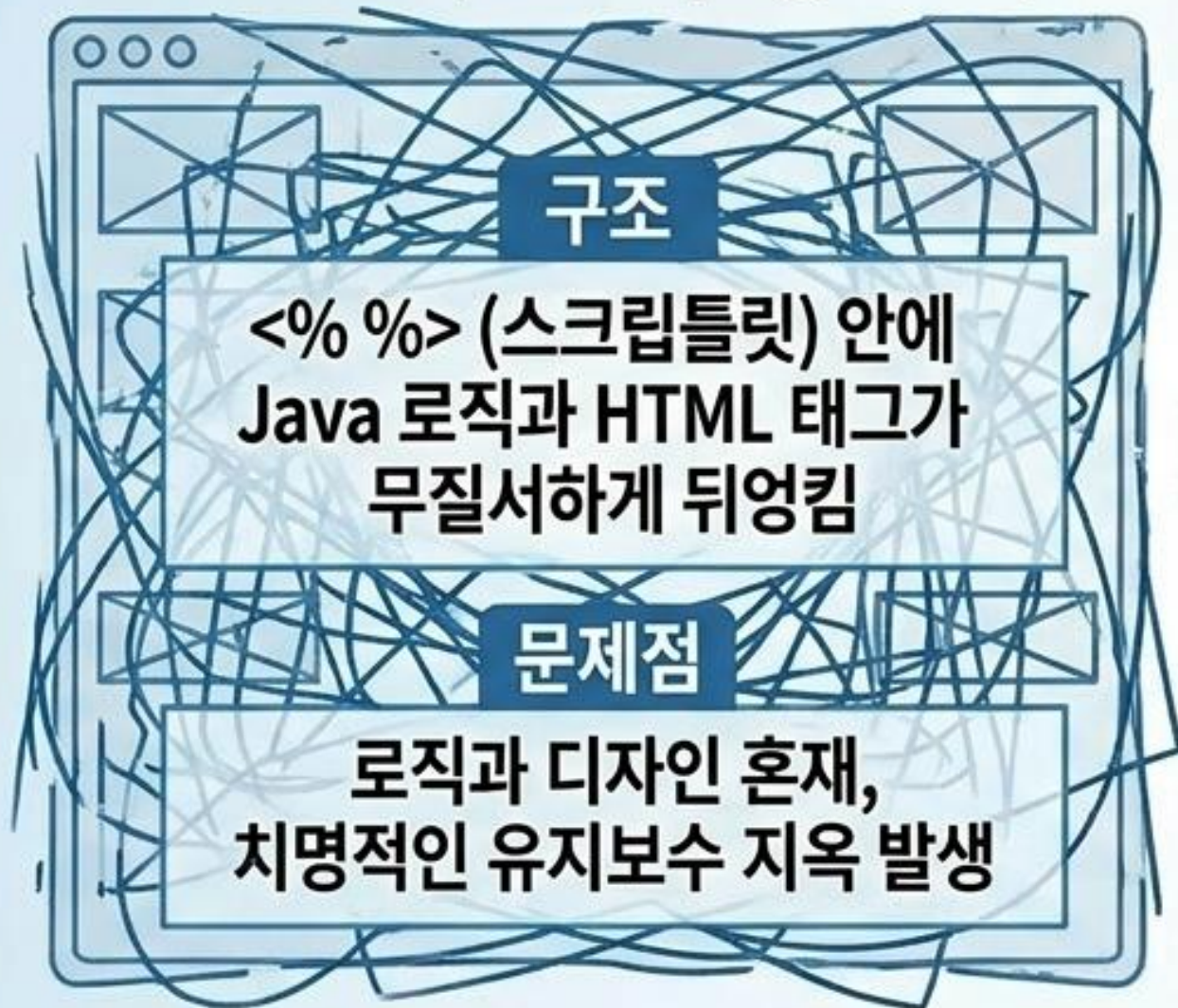
- ⚙️ 1. 전역 인코딩 처리: 모든 요청에 UTF-8 일괄 적용 (한글 깨짐 방지)
- 👤 2. 로그인 권한 인증: 세션 검증 후 미로그인 시 접근 차단 및 리다이렉트
- 📝 3. 공통 로깅 및 보안: 접근 로그 기록 및 악의적 요청 사전 검열



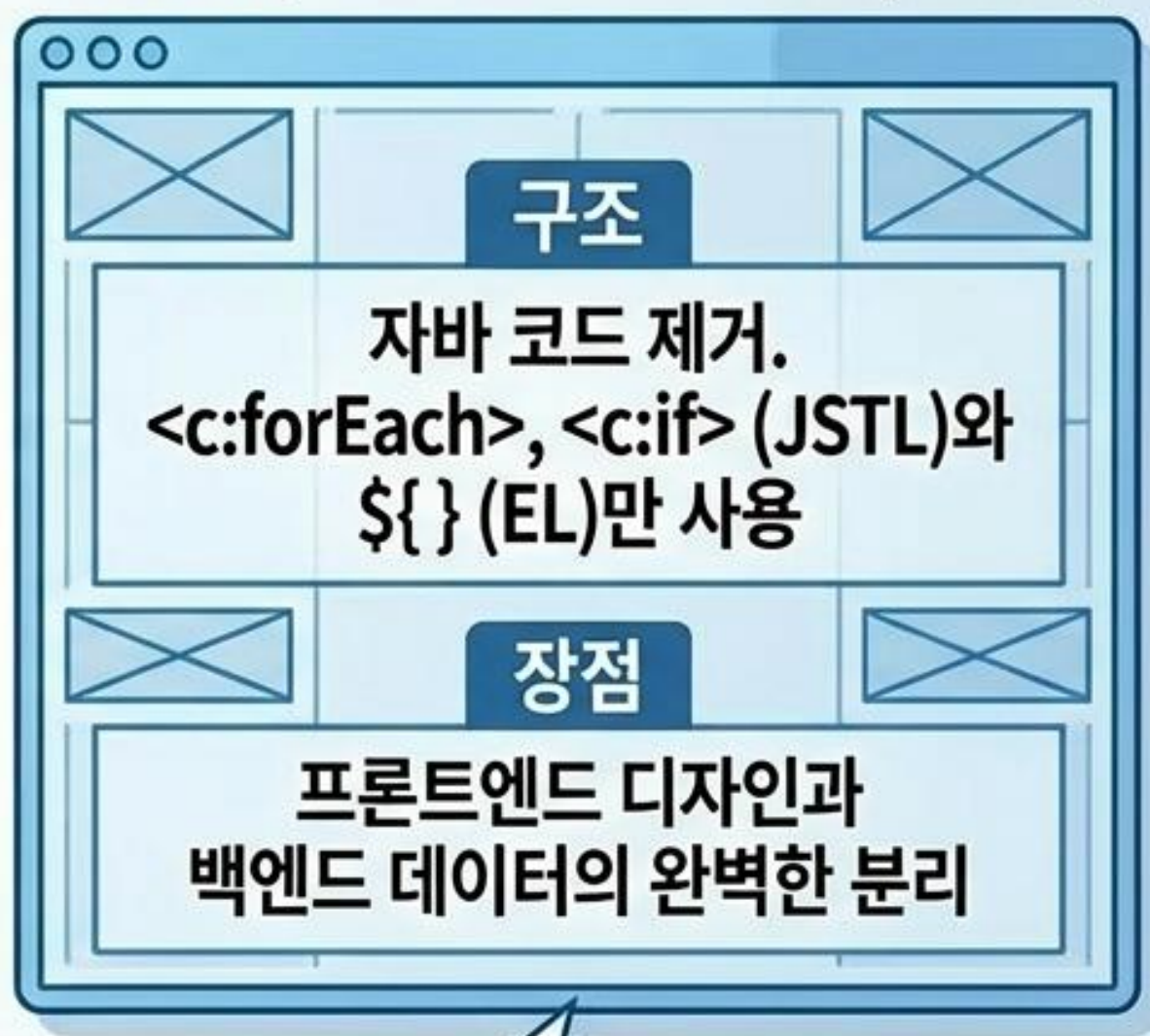
MVC 패턴에서 클라이언트 요청이 서블릿에 도달하기 전 중간에서 제어하는 핵심 방어선

뷰(View)의 진화: Scriptlet 지양과 JSTL/EL의 도입

Before (The Spaghetti)



After (The Clean Template)



EL의 자동 호출 마법

JSP에 `${user.id}`를 작성하면, 백엔드 로직에서 자바빈(JavaBean) 규약을 따라 DTO의 `user.getId()`를 자동으로 호출합니다.

실무 웹 아키텍처 절대 원칙 대시보드 (The Golden Rules)



JSP에서 DB 직접 호출 엄금

화면은 출력(View)만 담당하며, 반드시 Controller(Servlet)를 거쳐 MVC 패턴을 준수할 것.



Connection Pool 필수 도입

매번 DB 연결을 맺는 비용을 없애고 서버 다운을 막기 위해 DBCP/HikariCP를 활용할 것.



역순 자원 해제의 법칙 (finally 블록)

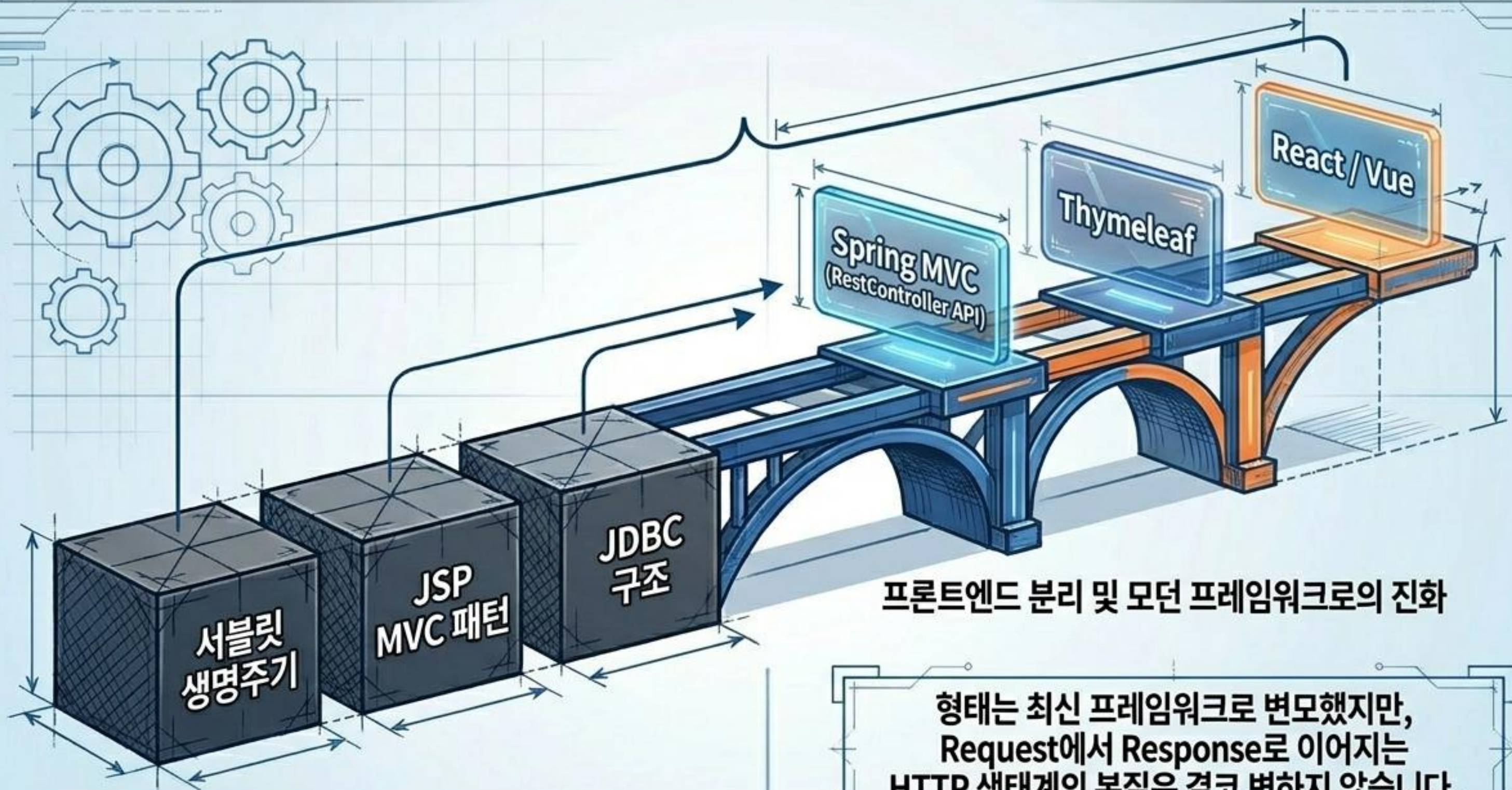
메모리 누수 방지를 위해 ResultSet → PreparedStatement → Connection 순으로 철저히 닫을 것.



DTO 내 비즈니스 로직 포함 금지

데이터 운반체인 DTO는 순수하게 데이터 필드만 유지하고, 핵심 연산은 Service 계층으로 이관할 것.

에필로그: 간단한 기초가 모던 아키텍처를 지탱한다



레거시 시스템, 공공/금융/SI 프로젝트의 확고한 표준